

Proyecto de Bases de Datos

**Análisis de datasets de reviews de
Amazon con MySQL, MongoDB y Neo4J**

Autores

Enrique Capella Magallón
María Belén Cao Sánchez

Madrid, Abril de 2026

Universidad Pontificia de Comillas – ICAI
Grado en Ingeniería Matemática e Inteligencia Artificial
Asignatura: Bases de Datos

ÍNDICE

1.	INTRODUCCIÓN	2
2.	CONFIGURACIÓN	3
3.	FLUJO DEL PROYECTO	4
3.1.	Carga de datos	4
4.	DISEÑO Y ALMACENAMIENTO DE LA BASE DE DATOS	5
4.1.	Datos estructurados en MySQL	5
4.2.	Datos en documentos en MongoDB	6
4.3.	Integración de ambos modelos	6
5.	ACCESO Y VISUALIZACIÓN DE LOS DATOS – PYTHON	7
5.1.	Evolución de reviews por año	7
5.2.	Evolución de la popularidad de los artículos	8
5.3.	Histograma por nota	8
5.4.	Evolución temporal de reviews	9
5.5.	Histograma de reviews por usuario	9
5.6.	Nube de palabras	10
5.7.	Gráfica adicional	10
6.	APLICACIÓN PYTHON – NEO4J	11
6.1.	Similitud entre usuarios	11
6.2.	Relación usuarios-artículos	12
6.3.	Usuarios con varios tipos de artículos	13
6.4.	Artículos populares y en común	13
7.	NUEVOS DATOS	15
7.1.	Procesamiento del nuevo dataset	15
7.2.	Inserción en MySQL	15
7.3.	Inserción en MongoDB	15
8.	VISUALIZACIÓN Y MACHINE LEARNING	16
8.1.	Visualización de datos con PowerBI	16
8.2.	Propuesta de modelo de Machine Learning	17
8.3.	Sistema de recomendación de artículos no consumidos	18
9.	CONCLUSIÓN	19

1. INTRODUCCIÓN

En este proyecto hemos trabajado con distintos datasets de reviews de Amazon con el objetivo de analizar la información en diferentes modelos de bases de datos.

Para ello, hemos utilizado principalmente MySQL, MongoDB y Neo4J, según el tipo de dato que trabajamos. MySQL se ha usado para almacenar información estructurada en tablas, MongoDB para el contenido en texto de las reseñas y Neo4J para analizar relaciones entre usuarios y productos mediante grafos.

Además, se ha desarrollado un código en Python que permite cargar los datos, consultarlos, visualizarlos y generar recomendaciones de productos en función del comportamiento de los usuarios.

Así, integramos distintas herramientas de forma que se complementan en un mismo sistema, mostrando cómo la combinación de información es eficaz para análisis como el desarrollado.

2. CONFIGURACIÓN

La configuración del proyecto ha sido determinada en el fichero *configuración.py*, donde se almacenan los parámetros de la conexión y acceso a las bases de datos en MySQL, MongoDB y Neo4j, así como las rutas de los datasets utilizados. Además, se incluyen otras variables auxiliares, como el número de usuarios considerados en el cálculo de similitudes en Neo4j.

Esto nos permite separar la lógica del programa de los parámetros preconfigurados, facilitando la adaptación del código a distintos entornos.

3. FLUJO DEL PROYECTO

El proyecto se ha dividido en varios módulos, cada uno con una funcionalidad, para mantener el código organizado y fácil de extrapolar.

La implementación se basa en varios ficheros principales:

- *configuracion.py*: contiene los parámetros de conexión a las bases de datos y las rutas de los datasets utilizados
- *load_data.py*: crea la base de datos y la carga inicial de los datos
- *inserta_dataset.py*: permite añadir nuevos conjuntos de datos sin necesidad de reconstruir toda la base
- *menu_visualizacion.py*: implementa un menú para generar las distintas gráficas
- *neo4JProyecto.py*: crea y gestiona el análisis de grafos en Neo4J
- *recomendador.py*: implementa el sistema de recomendación

Así, separamos el problema en partes independientes, facilitando la reutilización del proyecto.

3.1. Carga de datos

La carga de datos se realiza mediante el script *load_data.py*. Se crea la base de datos y las tablas en MySQL, y se inicializa la colección en MongoDB. Así, los ficheros JSON se procesan línea a línea, pudiendo trabajar con datasets grandes.

Conecta ambas bases mediante funciones específicas *conectar_mysql()* y la conexión con MongoClient. A continuación, se crean estructuras necesarias utilizando *crear_database_mysql()*, *crear_tablas_mysql()* y *crear_mongo_collection()*.

Una vez inicializada la estructura, los ficheros JSON se procesan línea a línea mediante la función *procesar_fichero()*, sin cargar el fichero completo en memoria, permitiendo usar datasets más grandes.

Dentro de *procesar_fichero()*, cada entrada del fichero pasa por varias funciones:

- *parsear_review_time(reviewTime)*: transforma la fecha a formato SQL
- *get_insert_category()*: inserta la categoría si no existe
- *get_insert_user()*: inserta el usuario si no existe
- *get_insert_product()*: inserta el producto si no existe
- *insert_review_mysql()*: inserta la parte estructurada
- *insert_review_mongo()*: inserta la parte textual

4. DISEÑO Y ALMACENAMIENTO DE LA BASE DE DATOS

Se ha adoptado un diseño para la carga y el almacenamiento de los datos que reparte la información entre MySQL y MongoDB según el tipo de dato. Para ello se ha desarrollado el fichero *load_data.py* que usa el fichero configuración.py para la conexión y ruta de los datasets.

El proceso consiste en conectarse a MySQL y MongoDB, crear la base de datos y tabla necesarias, procesar los ficheros JSON línea a línea, insertar la información estructurada en MySQL y el contenido de texto de reseñas en MongoDB.

4.1. Datos estructurados en MySQL

La base de datos relacional se ha diseñado para representar las entidades principales del sistema y sus relaciones, definiendo así cuatro tablas:

- Users: con reviewerID como clave primaria
- Categories: se genera un category_id interno
- Products: utiliza asin como identificador del producto
- Reviews: contiene claves foráneas hacia Users y Products

La tabla Reviews es el elemento central, conectando usuarios y productos mediante *reviewer_id* y *producto_id*, e incluyendo atributos como la puntuación, la fecha y la utilidad de la reseña.

El modelo está normalizado a 3FN, de la forma:

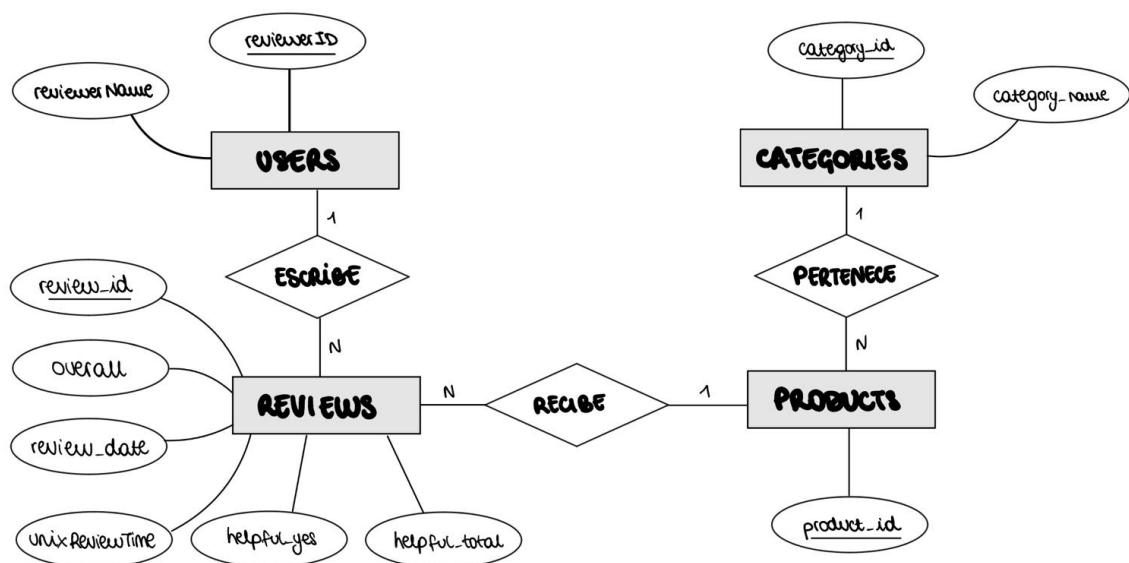


Figura 1: Diagrama entidad-relación de la base de datos en MySQL

De forma que el esquema relacional queda:

```
USERS (reviewerID, reviewerName)
CATEGORIES (category_id, category_name)
PRODUCTS (product_id, category_id)
          category_id FK REFERS TO CATEGORIES (category_id)
REVIEWS (review_id, reviewer_id, product_id, overall, review_date, unixReviewTime,
helpful_yes, helpful_total)
          reviewer_id FK REFERS USERS reviewerID)
          product_id FK REFERS PRODUCTS (product_id)
```

4.2. Datos en documentos en MongoDB

La información en forma de texto de las reseñas se almacena en MongoDB, en concreto, los campos *reviewText*, *summary* y *helpful*, mediante la función *insert_review_mongo()*. El campo clave del documento es *review_id*, usado como identificador común con SQL. Este identificador se genera en el momento de insertar la review en SQL y se usa en MongoDB, permitiendo la relación entre ambos.

Este tipo de datos presenta una estructura más flexible sin relaciones complejas, por lo que su almacenamiento en formato documental resulta más adecuado que uno relacional.

```
_id: ObjectId('69d8d1a95f07fe0b3a5d1b96')
review_id: 1
summary: "Magnetic board"
reviewText: "I like the item pricing. My granddaughter wanted to mark on it but I
w..."
▶ helpful: Array (2)
```

Figura 2: Ejemplo de documento JSON del dataset en MongoDB

4.3. Integración de ambos modelos

El sistema no usa MySQL y MongoDB independientemente, sino que se complementan.

Cada reseña (review) se divide en dos partes:

- MySQL: información estructurada (usuario, producto, puntuación, fecha)
- MongoDB: información textual (summary, reviewText)

Ambas partes se conectan mediante el campo *review_id*. Así, se pueden realizar consultas en SQL, analizar texto en MongoDB y combinarlos.

Ambas partes se relacionan mediante un identificador común, lo que permite reconstruir la información completa cuando es necesario, aprovechando las ventajas de cada modelo.

5. ACCESO Y VISUALIZACIÓN DE LOS DATOS – PYTHON

Se ha desarrollado un fichero *menú_visualizacion.py* que implementa un menú por terminal con opciones de visualización. El menú se repite hasta que el usuario seleccione la opción 0 de salida. En el código, el menú ofrece siete visualizaciones.

El fichero reutiliza la base de datos MySQL y MongoDB, con las funciones de conectar, una función para recuperar categorías *obtener_categorias()* y otra para pedir al usuario la categoría a analizar *pedir_categoria()*.

```
C:\Users\bcaos\OneDrive\Escritorio\2º IMAT\BB.DD\PROYECTO FINAL>C:/Users/bcaos/anaconda3/python.exe "c:/Users/bcaos/OneDrive/Escritorio/2º IMAT/BB.DD/PROYECTO FINAL/menu_visualizacion.py"

MENU DE VISUALIZACION
1. Evolucion de reviews por años
2. Evolucion de la popularidad de los articulos
3. Histograma por nota
4. Evolucion de las reviews a lo largo del tiempo
5. Histograma de reviews por usuario
6. Nube de palabras por categoria
7. Nota media por categoria
0. Salir
Seleccione una opcion: █
```

Figura 3: Menú de visualización de los datos desde la terminal Python

5.1. Evolución de reviews por año

Implementado en *mostrar_reviews_por_anos(categoria)*. Se ejecuta una consulta SQL agrupando por *YEAR(review_date)* y contando *reviews (COUNT(*))*, con posibilidad de filtrar por categoría.

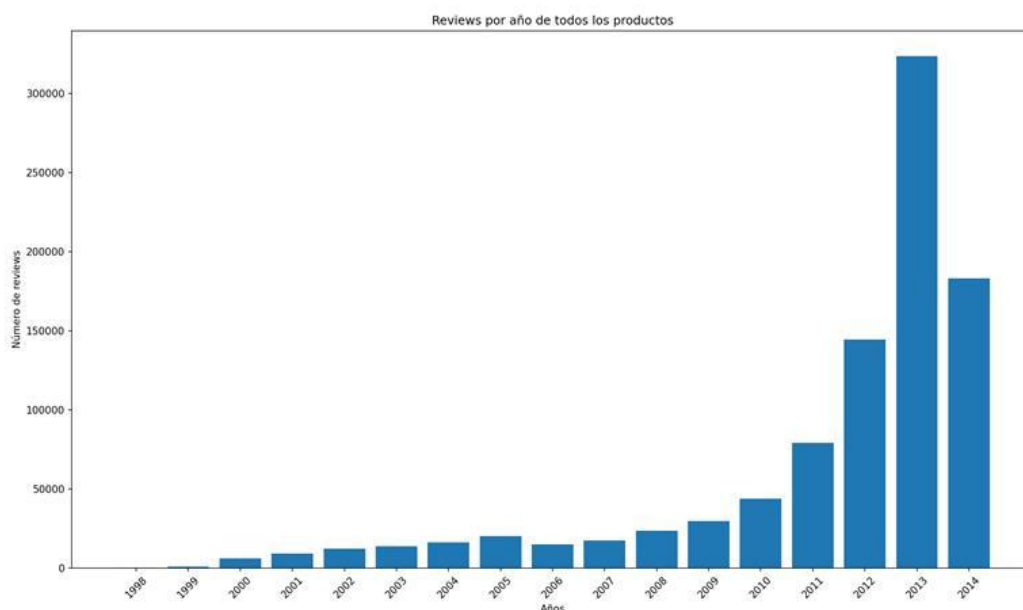


Figura 4: Evolución del número de reviews por año

5.2. Evolución de la popularidad de los artículos

Implementado en *mostrar_popularidad_articulos(categoria)*. Se agrupan los productos por *product_id*, contando reviews y ordenando de mayor a menor (*ORDER BY COUNT(*) DESC*). Muestra el ranking de popularidad.

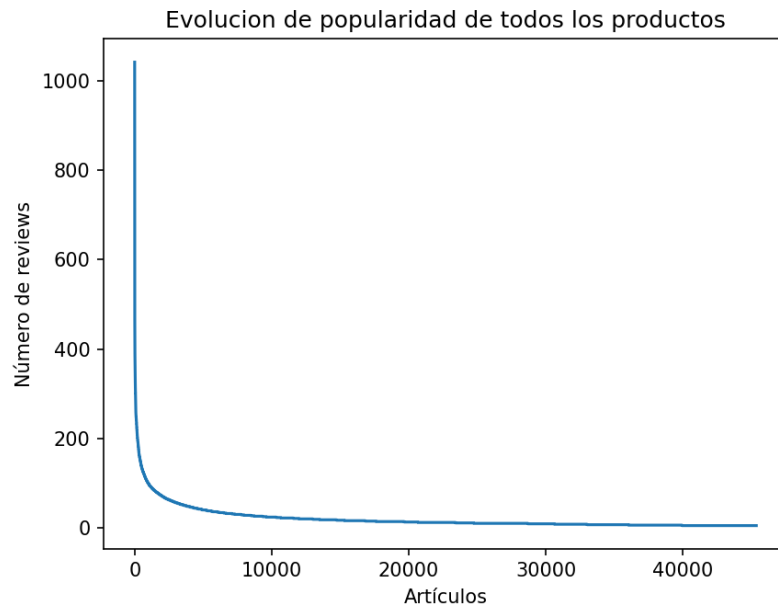


Figura 5: Ranking de popularidad de artículos por número de reviews

5.3. Histograma por nota

Implementado en *mostrar_histograma_por_nota(categoria)*. Se ejecuta una consulta SQL agrupando por overall y contando frecuencia. Permite ver la distribución de las puntuaciones.

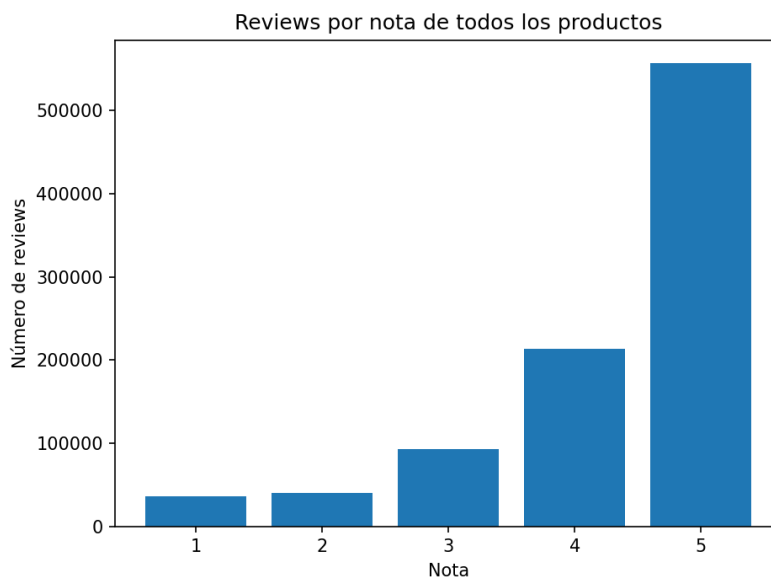


Figura 6: Distribución de puntuaciones de las reviews

5.4. Evolución temporal de reviews

Implementado en *mostrar_evolucion_reviews_tiempo()*. Se obtienen *unixReviewTime* y categorías desde MySQL, y en Python se construyen curvas acumuladas por categoría. Permite comparar el crecimiento entre categorías.

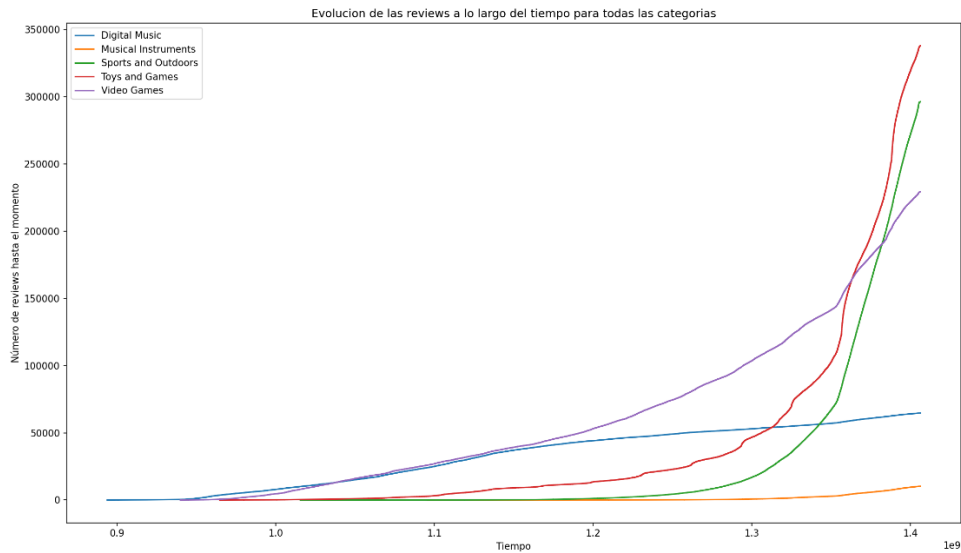
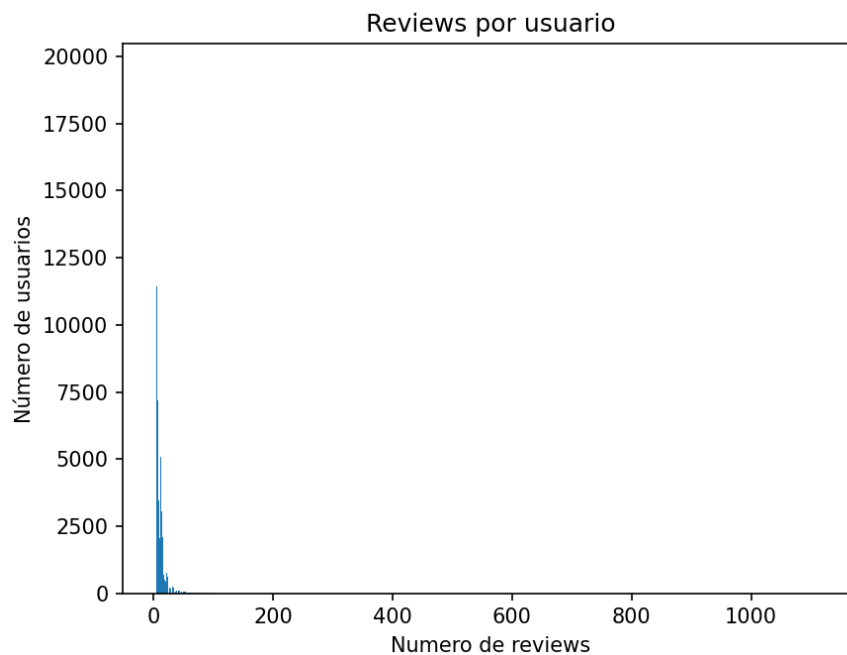


Figura 7: Evolución acumulada de reviews a lo largo del tiempo por categoría

5.5. Histograma de reviews por usuario

Implementado en *mostrar_histograma_reviews_por_usuario()*. Se usa una subconsulta SQL para contar reviews por usuario y luego agrupar por ese valor. Permite analizar el comportamiento de los usuarios.



5.6. Nube de palabras

Es un ejemplo claro de integración MySQL y MongoDB mediante `review_id`. Implementado en `mostrar_nube_palabras_categoria(categoría)`. Se obtiene `review_id` desde MySQL, se recupera `summary` desde MongoDB, se filtran palabras de longitud determinada y se genera la nube con *WordCloud*.



Figura 9: Nube de palabras generada a partir de los summaries de las reviews

5.7. Gráfica adicional

Hemos intentado resumir el comportamiento global de cada categoría, tal que implementamos el cálculo de la nota media por categoría en *mostrar_nota_media_por_categoria()*, con *AVG(overall)* y representándolo con barras.

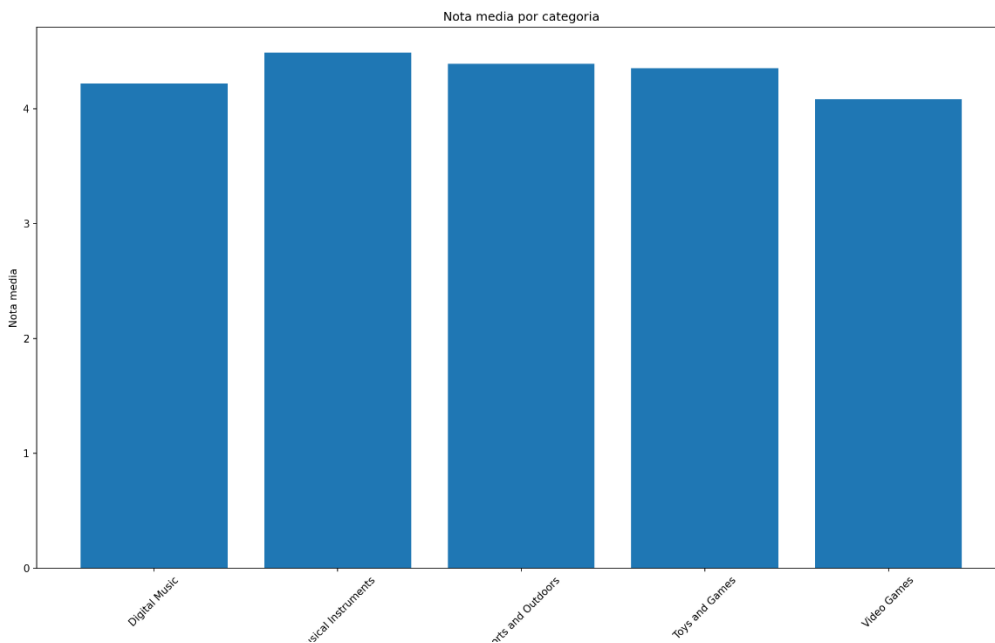


Figura 10: Nota media por categoría

6. APLICACIÓN PYTHON – NEO4J

Se ha desarrollado el fichero *neo4JProyecto.py*, que permite analizar las relaciones entre usuarios y productos utilizando un modelo de grafos en Neo4J.

El programa funciona mediante un menú interactivo desde terminal, donde cada opción corresponde a uno de los subapartados. Antes de ejecutar cada opción, se limpia la base de datos de Neo4J, para garantizar que cada análisis parte de un grafo independiente.

```
C:\Users\bcaos\OneDrive\Escritorio\2º IMAT\BB.DD\PROYECTO FINAL>C:/Users/bcaos/anaconda3/python.exe "c:/Users/bcaos/OneDrive/Escritorio/2º IMAT/BB.DD/PROYECTO FINAL/neo4JProyecto.py"

MENÚ NEO4J
0. Salir
1. Similitud entre usuarios
2. Enlaces entre usuarios y artículos
3. Usuarios y tipos de artículo
4. Artículos populares y artículos en comun
Seleccione una opción: █
```

Figura 11: Menú interfaz por terminal del menú de Neo4J

6.1. Similitud entre usuarios

Seleccionamos los usuarios con mayor número de reviews (controlado mediante una variable en *configuracion.py*) y se calcula la similitud entre ellos.

Para ello, primero se obtienen sus valoraciones desde MySQL y se almacenan en diccionarios. Luego se calcula la similitud utilizando la función *pearson_correlation()*, que compara las puntuaciones en productos comunes.

En Neo4J se crean nodos User y relaciones SIMILAR entre aquellos usuarios cuya similitud es válida. Esto permite identificar usuarios con gustos parecidos.



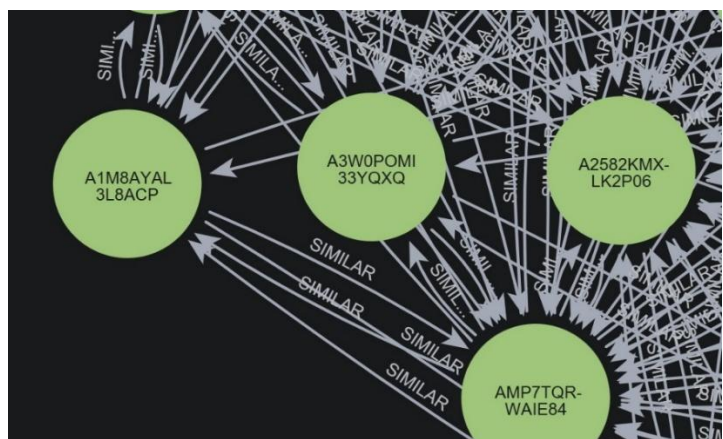


Figura 12: Grafo de similitud entre usuarios (y ampliado)

6.2. Relación usuarios-artículos

Se seleccionan artículos de una categoría determinada, introducida por el usuario, y se obtiene un conjunto de artículos aleatorios.

A partir de ellos, se recuperan todas las reviews asociadas y se construye un grafo con los nodos Usuario, Artículo y relaciones PUNTUO, que incluyen propiedades como la puntuación (*overall*) y la fecha (*review_date*). Así, se ven qué usuarios han interactuado con cada producto.

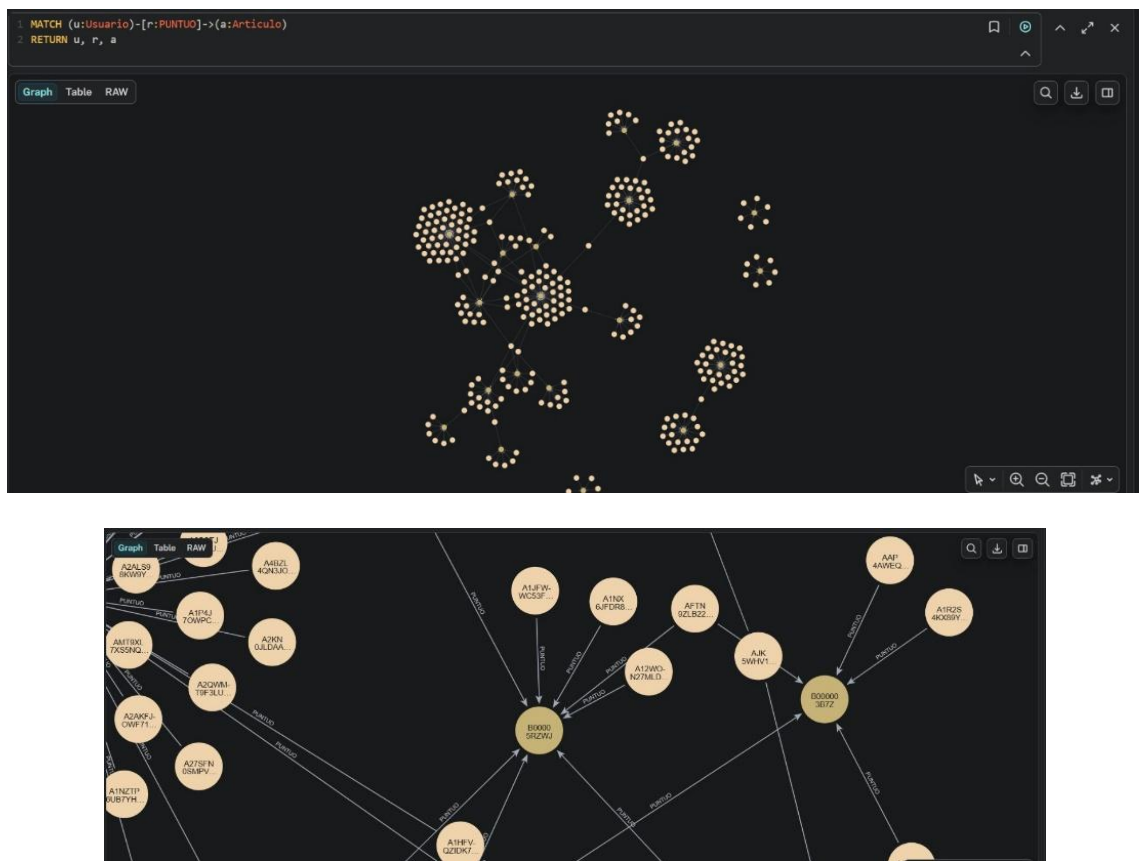


Figura 23: Grafo de relaciones entre usuarios y artículos (y ampliado)

6.3. Usuarios con varios tipos de artículos

Se seleccionan los primeros usuarios ordenados por nombre y se analizan las categorías de los productos que han valorado. A partir de ahí, se filtran aquellos usuarios que han interactuado con al menos dos tipos distintos de artículos.

En Neo4J se construye un grafo con los nodos Usuario, TipoArticulo y las relaciones CONSUME, que incluyen la cantidad de artículos consumidos de ese tipo. Así, se permite analizar la diversidad de intereses de los usuarios.

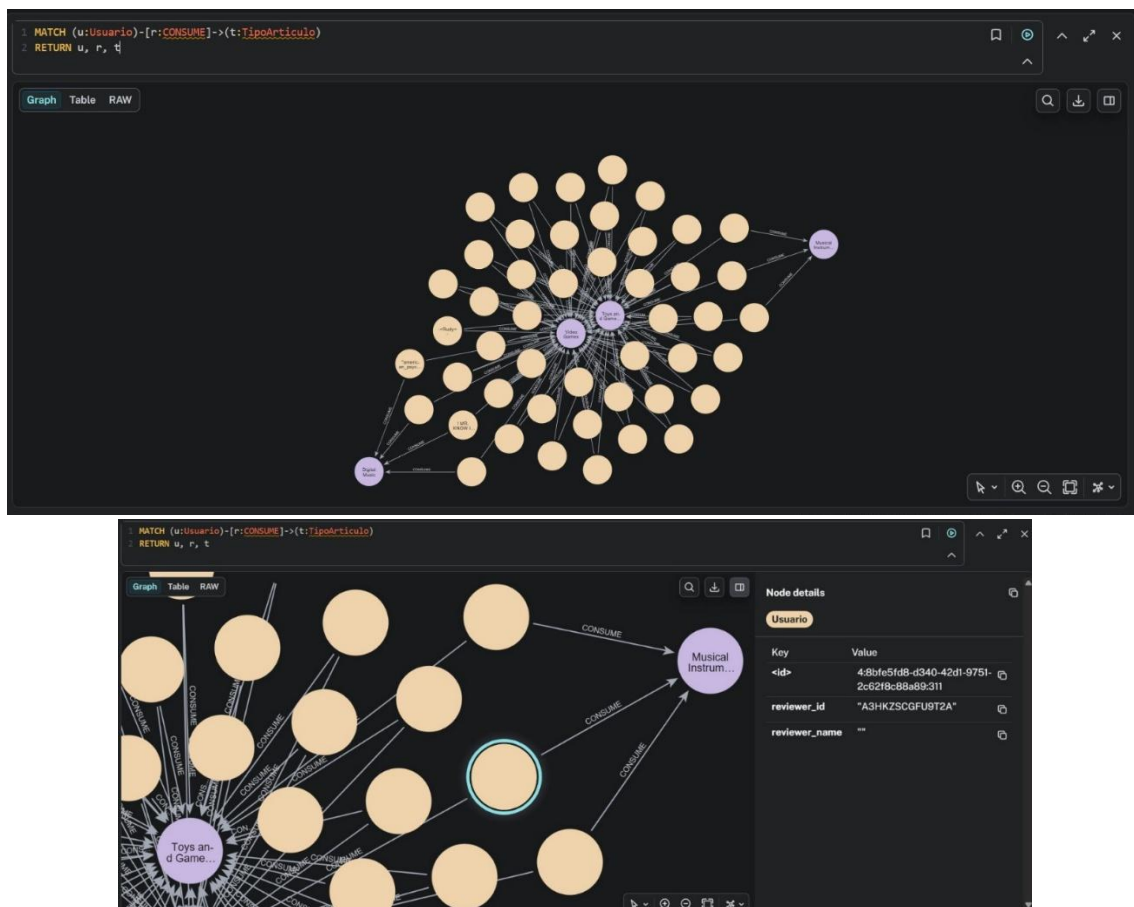


Figura 14: Grafo de usuarios y tipos de artículos consumidos (y ampliado)

6.4. Artículos populares y en común

Seleccionamos los artículos más populares con menos de 40 reviews. Se construye un grafo con los nodos Artículo (incluyendo el número de reviews), Usuario y las relaciones PUNTUO.

Además, se añaden relaciones EN_COMUN entre usuarios, indicando cuántos artículos han puntuado en común. Esto permite detectar conexiones entre usuarios basadas en intereses compartidos.

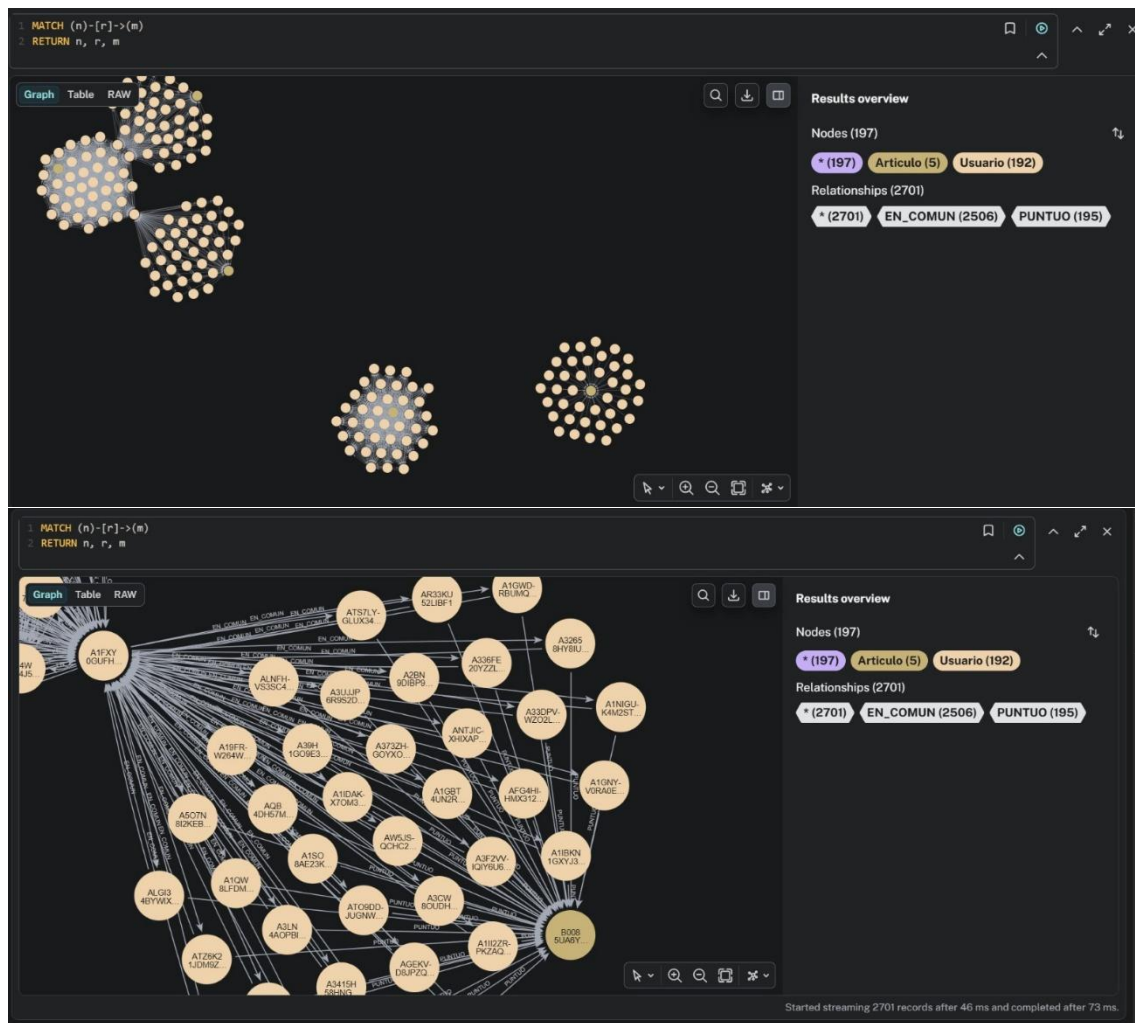


Figura 15: Grafo de artículos populares y relaciones en común entre usuarios (y ampliado)

7. NUEVOS DATOS

En este apartado se ha implementado un mecanismo para insertar nuevos datasets sin necesidad de reconstruir la base de datos completa. Para ello, se ha desarrollado el fichero *inserta_dataset.py*.

El objetivo es poder añadir información adicional teniendo en cuenta que un mismo usuario puede haber realizado reviews en distintas categorías de productos, es decir, usando cualquier otro dataset que se nos proporcione.

El funcionamiento es similar al de *load_data.py*, reutilizando muchas de sus funciones.

7.1. Procesamiento del nuevo dataset

El fichero JSON elegido es el referente a Outdoor Sports y se procesa mediante una función equivalente a *procesar_fichero()*, evitando cargar todo el dataset en memoria.

Cada registro contiene campos como *reviewerID*, *asin*, *overall*, *reviewText*, *summary* o *reviewTime*. Esto permite tratar el nuevo dataset de la misma forma que los anteriores.

7.2. Inserción en MySQL

Para evitar duplicados, se reutilizan las funciones ya definidas:

- *get_insert_user()*: comprueba si el usuario existe antes de insertarlo
- *get_insert_product()*: inserta el producto solo si no está registrado
- *get_insert_category()*: permite añadir nuevas categorías si aparecen

Finalmente, la función *insert_review_mysql()* inserta la review con sus atributos, garantizando no duplicar información.

7.3. Inserción en MongoDB

El contenido en formato texto (*reviewText*, *summary*, *helpful*) se almacena en MongoDB mediante la función *insert_review_mongo()*.

Se mantiene el uso del campo *review_id* como identificador común, permitiendo seguir relacionando ambas bases e integrar nuevos datos sin alterar el modelo existente.

7.4. Integración de usuarios en múltiples categorías

Cabe destacar que un mismo *reviewerID* puede aparecer en distintos datasets. Gracias al uso de *get_insert_user()*, el usuario se almacena una única vez y se relaciona con múltiples productos y categorías, aportando más información útil al sistema.

8. VISUALIZACIÓN Y MACHINE LEARNING

Incorporamos herramientas de visualización y una aproximación a sistemas de recomendación. Para ello, se utilizan los datos almacenados para generar nuevas gráficas en Power BI y se plantea un modelo de Machine Learning orientado a recomendar productos a los usuarios.

8.1. Visualización de datos con PowerBI

Para visualizar mejor las gráficas realizadas en Python, hemos utilizado PowerBI como herramienta externa para la representación de las mismas.

Primero hemos establecido la conexión con la base de datos MySQL creada en el proyecto, hemos cargado las tablas necesarias y hemos generado distintas visualizaciones.

- Número de reviews por año (Figura 16): usamos el campo *review_date* y vemos su evolución temporal
- Distribución de puntuaciones (Figura 17): representa la frecuencia de cada valor de *overall*
- Nota media por categoría (Figura 18): combina tablas de productos y categorías

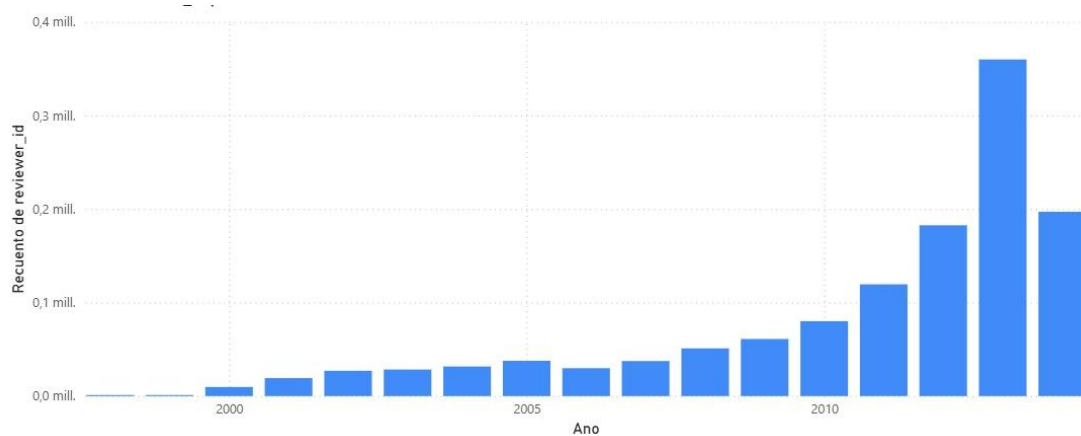


Figura 15: Evolución de reviews por año en PowerBI

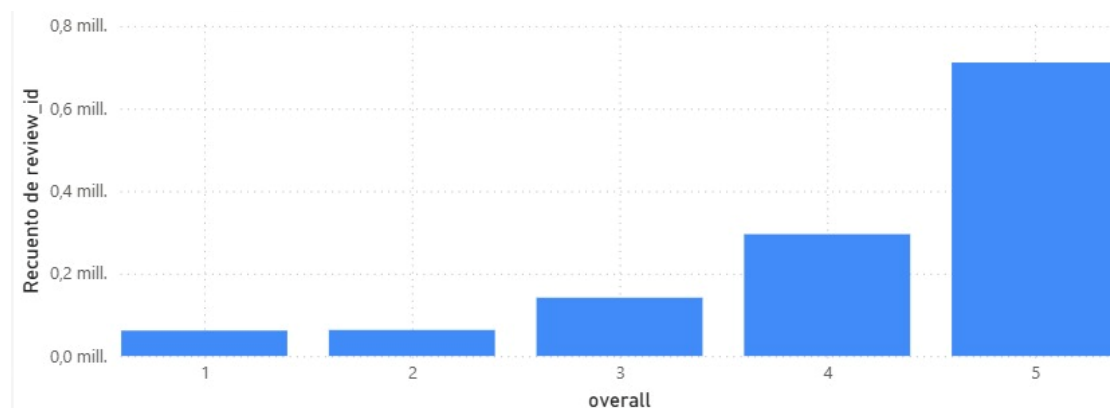


Figura 16: Distribución de reviews por nota en PowerBI

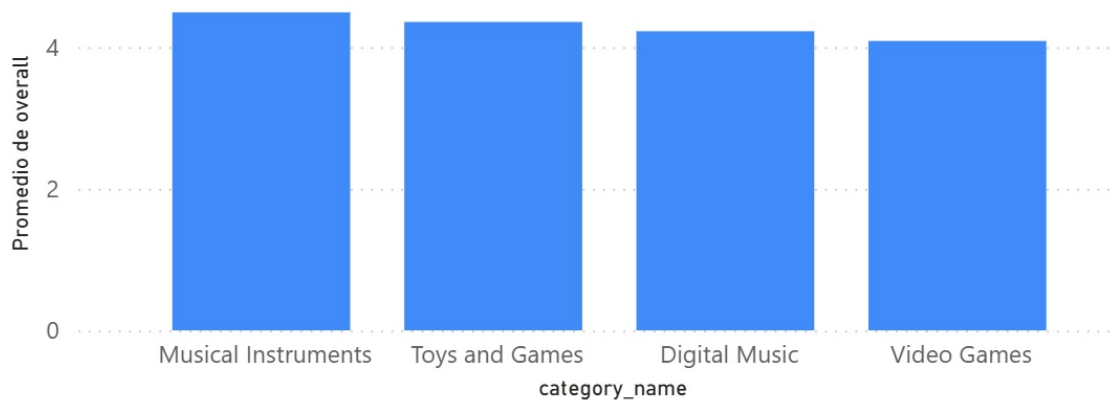


Figura 17: Nota media por categoría en PowerBI

8.2. Propuesta de modelo de Machine Learning

A partir de los datos almacenados en la base de datos, construimos un sistema de recomendación que sugiera productos a los usuarios que todavía no han consumido haciendo uso de un sistema de Machine Learning.

Para ello, utilizaríamos principalmente la tabla Reviews, ya que contiene la relación entre usuarios (*reviewer_id*) y productos (*product_id*), junto con la puntuación (*overall*). Con esta información se puede crear un dataset donde cada fila sea un usuario y cada columna un producto, y en cada intersección esté la nota que ha dado.

Proponemos KNN (k-Nearest Neighbors) como modelo por su sencilla implementación, que consiste en encontrar usuarios con comportamientos similares en función de sus valoraciones y utilizar esa información para generar recomendaciones.

El funcionamiento sería:

- Representamos a cada usuario según las puntuaciones que ha dado
- Comparamos usuarios entre sí para ver cuáles son más parecidos (usando distancia euclídea, por ejemplo)
- Para un usuario concreto, buscamos sus k “vecinos” más cercanos (usuarios más similares)
- Miramos qué productos han valorado esos vecinos que el usuario aún no
- Recomendamos de esos productos, los diez que mejor valoración media tienen entre vecinos

Esto nos recuerda a lo que ya hemos hecho en Neo4J, donde también calculamos similitudes entre usuarios.

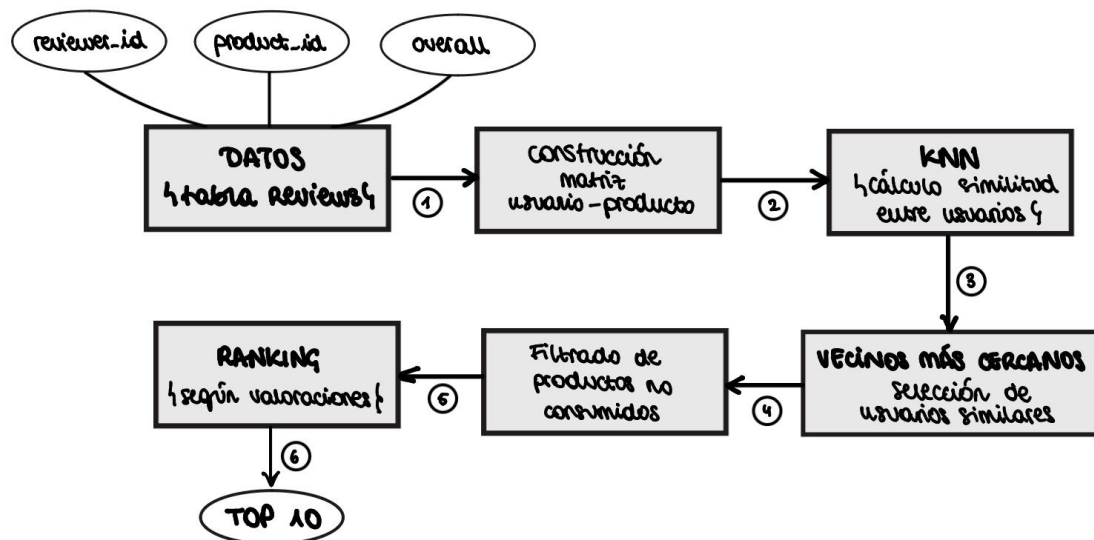


Figura 18: Diagrama de flujo del funcionamiento del modelo de ML propuesto

8.3. Sistema de recomendación de artículos no consumidos

Se ha implementado un sistema de recomendación en el fichero *recomendador.py*, cuyo objetivo es sugerir productos al usuario en función de su comportamiento previo.

El sistema funciona a partir de un identificador de usuario (*reviewerID*) y una categoría seleccionada por el usuario. Primero comprobamos si el usuario existe en la base de datos mediante la función *usuario_existe()*. Luego, se muestran las categorías disponibles utilizando *obtener_categorias()* y se solicita al usuario una categoría concreta.

Una vez introducido, el sistema genera recomendaciones mediante la función *obtener_recomendaciones()*, que devuelve los artículos más populares de la categoría seleccionada que el usuario aún no ha valorado.

```

PS C:\Users\enriq\OneDrive\Desktop\Bases de datos\PrFINALBEL> & C:\Users\enriq\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/enriq/OneDrive/Desktop/Bases de datos/PrFINALBEL/src/ejercicio5.py"
RECOMENDADOR

Introduce el reviewerID: AIXZKN4ACSKI

Categorías disponibles:
  · Digital Music
  · Musical Instruments
  · Sports and Outdoors
  · Toys and Games
  · Video Games

Introduce la categoría: Sports and Outdoors

Top 10 artículos recomendados:
1. B001HBHNHE (1042 reviews)
2. B001T7QJ90 (763 reviews)
3. B000S50DN2 (647 reviews)
4. B00100748Q (513 reviews)
5. B0000C50K3 (427 reviews)
6. B002ZYRV2E (401 reviews)
7. B0020KWHVO (398 reviews)
8. B000GCRMCG (393 reviews)
9. B001HBHNHY (372 reviews)
10. B0035L35AB (359 reviews)
  
```

Figura 16: Ejecución del recomendador en terminal

9. CONCLUSIÓN

Finalmente hemos desarrollado un sistema de gestión y análisis de datos a partir de datasets de reviews de Amazon, combinando diferentes bases de datos y herramientas de análisis.

Por un lado, se ha diseñado un modelo que combina MySQL y MongoDB, separando los datos estructurados de los no estructurados y permitiendo trabajar con ambos tipos de información. Por otro lado, el uso de Neo4J ha permitido analizar relaciones entre usuarios y productos, aportando la visión por grafos.

Además, se ha implementado un flujo en Python que integra todas estas partes, facilitando la carga de datos, su visualización y la generación de recomendaciones. También se ha extendido el sistema a nuevos datasets sin necesidad de reconstruir la base de datos.

Finalmente, se ha planteado un modelo de Machine Learning y se ha implementado un sistema de recomendación al usuario, mostrando el valor añadido que aporta el almacenamiento de los datos.